

Actividad – Reto 2

Descripción del Reto

Contexto del Reto

Estás a cargo del diseño e implementación de un sistema de alerta temprana para una flota vehicular. Cada vehículo envía constantemente su **posición geográfica** y su **estado de operación** a través de un endpoint. En situaciones críticas, como cuando el conductor presiona un **botón de pánico** o los sensores detectan una anomalía, se envía un **evento de tipo "Emergency"**.

Tu objetivo es garantizar que:

- **Se reciban y procesen 1000 eventos correctamente.**
- **Se envíe una alerta por correo electrónico a una cuenta Gmail específica** en el momento en que se reciba un evento de tipo "Emergency".
- **Se cumplan las restricciones técnicas dadas** (límite de peticiones, instancias máximas de procesadores, etc.).

• Objetivo

- **Implementar una solución arquitectónica** que permita recibir los eventos de los vehículos en tiempo real.
- **Detectar eventos de emergencia** en el flujo continuo de datos.
- **Enviar una notificación por correo electrónico** a la cuenta Gmail configurada en menos de 30 segundos desde la recepción del evento.

Actividad – Reto 2

Descripción del Reto

Requerimientos Funcionales

- **Recepción de Eventos:**
 - Diseñar un endpoint que pueda recibir 1000 solicitudes en 30 segundos.
 - Garantizar que el 100% de las solicitudes sean procesadas correctamente.
- **Detección de Emergencias:**
 - Identificar eventos con el tipo "Emergency" dentro del flujo continuo de datos.
 - Registrar un **log** con la fecha y hora exacta de la recepción de un evento de tipo emergencia.
- **Notificación por Correo Electrónico:**
 - Enviar un correo a una cuenta Gmail configurada cuando se detecte un evento de tipo "Emergency".
 - Registrar un **log** con la fecha y hora del envío del correo.

Actividad – Reto 2

Descripción del Reto



Requerimientos No Funcionales

- **Tasa de Peticiones:**
 - Si se utiliza un API Gateway, la tasa máxima debe ser de **15 peticiones por segundo** (rate).
 - El valor de **burst** puede quedar en su configuración predeterminada.
- **Capacidad de Procesamiento:**
 - Si se utilizan procesadores como **Docker, Lambda, ALB, EC2, ECS**, el límite máximo es de **10 instancias** activas simultáneamente.
- **Logs:**
 - Se deben incluir registros claros con la hora exacta de:
 - La recepción del evento de tipo "Emergency".
 - El envío exitoso del correo de alerta.
- **Tiempo de Entrega del Correo:**
 - Si el correo llega en **menos de 15 segundos: 2.5 puntos**.
 - Si el correo llega entre **15 y 45 segundos: 1.5 puntos**.
 - Si el correo llega después de **45 segundos: 0.5 puntos**.
 - **El tiempo total se medirá entre el último envío realizado en el k6 vs la hora de envío del correo y recepción del mismo.**
- **Estándar para el Correo:**
 - El correo debe enviarse a una cuenta **Gmail personal** configurada por el estudiante.
 - El contenido del correo debe ser claro e indicar que se recibió un evento de tipo "Emergency".

Actividad – Reto 2

Descripción del Reto

Entregables

- **Código Fuente:**
 - Debe incluir toda la implementación necesaria para recibir eventos, detectar emergencias y enviar correos.
- **Documentación Técnica:**
 - **Decisiones de Arquitectura:** Justificar por qué se eligieron los componentes y servicios utilizados.
 - **Atributo de Calidad más Importante:** Explicar cuál es y por qué fue priorizado.
 - **Diagrama de la Arquitectura:** Representación visual clara de la solución.
 - **Tácticas de Arquitectura:** Explicar las tácticas utilizadas para cumplir con los objetivos.
- **Logs de Ejecución:**
 - Un archivo o consola que muestre claramente los logs de:
 - Recepción del evento de tipo "Emergency".
 - Envío exitoso del correo electrónico.
- **Video de Explicación de la solución para los estudiantes que no pueden asistir al encuentro sincrónico, donde se evidencia, se explique y se detalle como crearon la arquitectura y todos los detalles del documento técnico e implementación. Para los estudiantes que asisten al encuentro sincrónico, deben generar una presentación detallando toda la implementación y mostrando el demo en vivo.**

Actividad – Reto 2

Descripción del Reto



Escenario de Pruebas

- Se proporcionará un **script de k6** para enviar **1000 peticiones en 30 segundos** al endpoint configurado por el estudiante.
- El script simula vehículos que envían eventos de tipo "Position" y "Emergency".
- Se debe garantizar que el sistema detecte correctamente el evento "Emergency" y ejecute la acción de notificación.



Restricciones

- No se pueden superar las **15 peticiones por segundo** en el API Gateway.
- Los procesadores de peticiones (Lambda, Docker, ECS, etc.) no pueden exceder las **10 instancias simultáneas**.
- Se debe usar una cuenta **Gmail** para la notificación.
- El envío de correos debe ser **consistente y medible**.

Actividad – Reto 1

Rúbrica de Presentación
(2.5 puntos)

Criterio	Ponderación
Justificación de Decisiones de Arquitectura	0.5
Atributo de Calidad más Importante	0.5
Diagrama de Arquitectura	0.5
Tácticas de Arquitectura	1.0
Subtotal Documentación Técnica:	2.5
Tiempo de Entrega del Correo (<15s)	2.5
Tiempo de Entrega del Correo (15-45s)	1.5
Tiempo de Entrega del Correo (>45s)	0.5
Subtotal Demostración en Vivo:	2.5

Actividad – Reto 2

Descripción del Reto



Configuraciones Solicitadas

A nivel de Api Gateway:

Stages

Stage actions ▼

Create stage

Stage details [Info](#)

Edit

Stage name
prod

Cache cluster [Info](#)
⊖ Inactive

Default method-level caching
⊖ Inactive

Rate [Info](#)
15

Burst [Info](#)
2000

Web ACL
-

Client certificate
-

Payload Ejemplo:

```
{
  "type": "Position",
  "vehicle_plate": "ABC-123",
  "coordinates": {
    "latitude": 12.345,
    "longitude": 67.890
  },
  "status": "OK"
}
```

Correo Ejemplo:

Alerta de Emergencia

d.alejor10@gmail.com a través de amazonses.com

para mí ▼

Traducir al español

Alerta de Emergencia

Placa: VFH-800

Estado: OK

Evento: Emergency

Responder

Reenviar

Actividad – Reto 2

Descripción del Reto



Configuraciones Solicitadas

Salida espera del script de k6:

```
C:\> Seleccionar C:\WINDOWS\system32\cmd.exe

Grafana
K6

execution: local
script: k6-script.js
output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 1m0s max duration (incl. graceful stop):
* default: 1000 iterations shared among 10 VUs (maxDuration: 30s, gracefulStop: 30s)

is status 200

checks.....: 100.00% 1000 out of 1000
data_received.....: 533 kB 19 kB/s
data_sent.....: 199 kB 7.1 kB/s
http_req_blocked.....: avg=1.28ms min=0s med=0s max=130.35ms p(90)=0s p(95)=0s
http_req_connecting.....: avg=42.62µs min=0s med=0s max=4.93ms p(90)=0s p(95)=0s
http_req_duration.....: avg=176.19ms min=92.81ms med=114.05ms max=359.38ms p(90)=261.51ms p(95)=272.08ms
{ expected_response:true }...: avg=176.19ms min=92.81ms med=114.05ms max=359.38ms p(90)=261.51ms p(95)=272.08ms
http_req_failed.....: 0.00% 0 out of 1000
http_req_receiving.....: avg=120.26µs min=0s med=0s max=1.65ms p(90)=523.8µs p(95)=966.03µs
http_req_sending.....: avg=102.03µs min=0s med=0s max=1.04ms p(90)=547.17µs p(95)=613.33µs
http_req_tls_handshaking.....: avg=340.88µs min=0s med=0s max=35.9ms p(90)=0s p(95)=0s
http_req_waiting.....: avg=175.97ms min=92.81ms med=114ms max=359.38ms p(90)=261.31ms p(95)=272.06ms
http_reqs.....: 1000 35.691471/s
iteration_duration.....: avg=278.23ms min=193.43ms med=215.16ms max=568.65ms p(90)=362.55ms p(95)=373.34ms
iterations.....: 1000 35.691471/s
vus.....: 1 min=1 max=10
vus_max.....: 10 min=10 max=10

Running (0m28.0s), 00/10 VUs, 1000 complete and 0 interrupted iterations
default [=====] 10 VUs 28.0s/30s 1000/1000 shared iters

E:\k6>
```